



Title: A cleaner architecture where the source of truth is in the

database

Author: Jeff Meridian

This document provides a detailed overview of the `jeffmeridianlocal_sanity.spec.ts` test suite, designed to help coding agents understand the architecture, mocking strategies, and functional coverage of the local environment tests.

Overview

The `jeffmeridianlocal_sanity.spec.ts` is a comprehensive Playwright-based test suite that validates the local development environment for the Writer Studio and its associated Elevanty-generated website. It serves as both a UI sanity check and a backend tool integration verification.

Primary Target: `http://localhost:8080`

Maintenance Notes for Agents

- **Adding New Plugins:** If adding a new plugin to `anti/plugin/`, ensure it supports the `TESTDBFILE` environment variable to maintain compatibility with this suite.
- **Schema Changes:** If the database schema evolves, update the `sql` string in the `top-level test.beforeAll` block to include new tables or columns.
- **Cleanup:** The suite automatically unlinks the `testmock*.db` files after execution. Do not store persistent data in these files.

Prerequisites

To run these tests successfully, the following must be available on the system:

1. **Node.js & Playwright:** `npx playwright test`
1. **Python 3:** For executing the plugins and `plugin_host.py`.
1. **SQLite3 CLI:** Must be installed and available in the `$PATH` for the mock database seeding.
1. **Local Server:** The Eleventy dev server should be running at `http://localhost:8080`.

Running the Tests

To run the full suite:

```
npx playwright test transcripts/tests/
jeff_meridian_local_sanity.spec.ts
```

To run only the Python tool integration tests:

```
npx playwright test -g "Anti Plugin
Tools|Writer Studio Tools API"
```

Core Architecture

1. Mocking Strategy (SQLite Isolation)

To ensure tests are non-destructive and idempotent, the suite uses **Database-Level Isolation**.

- **Worker-Specific Databases:** At the start of the test run (`test.beforeAll`), each Playwright worker generates a unique SQLite file (e.g., `testmock0.db`).
- **Seeding:** The mock database is seeded using the `sqlite3` CLI with a standardized set of data:
 - A `documents` table with a “Mock Doc” (ID 999).
 - An `app_state` table with the active document state set to the “Mock Doc”.
- **Environment Injection:** The path to this mock database is passed to all child processes (Python scripts and the MCP host) via the `TESTDBFILE` environment variable.

1. Test Grouping

The suite is divided into several logic blocks:

D. Writer Studio Tools API (MCP Integration) Validates the Python-based MCP (Model Context Protocol) server. • Process Spawning: It spawns <code>plugin_host.py</code> as a child process. • JSON-RPC Communication: Communicates with the host over <code>stdin/stdout</code> using standard JSON-RPC payloads. • Tool Coverage: Exercises the searchdocuments, batchoperations, and <code>replace_text</code> tools against the mock database. E. Anti Plugin Tools (Direct Execution) Directly tests the standalone Python plugins used for active document management. • Verifies that <code>readactivatedocument.py</code>, <code>editactivatedocument.py</code>, <code>replaceactivatedocumentselectedtext.py</code> correctly interact with the SQLite <code>app_state</code> table. • Serialization: This block uses <code>test.describe.serial</code> to prevent concurrent write collisions on the worker's mock database.	A. Homepage Sanity & Section Visibility Validates that the Eleventy build is correct and all major UI sections are present. • Generic Testing: Instead of looking for specific article titles, it verifies the existence of structural classes like <code>.hero-featured</code>, <code>.app-icon-card</code>, and <code>.post-card</code>. • Resilience: Tests verify that there is <i>at least one</i> item in each grid, ensuring the test doesn't fail just because a specific article was renamed or removed. B. Language Switching & Persistence Tests the internationalization (i18n) logic. • Verifies that clicking flags updates the URL (e.g., <code>/?lang=de</code>). • Validates that the language preference is persisted in <code>localStorage</code> and correctly applied upon a page reload. C. Robust Navigation (Games & Blog) Tests the link integrity across the site. • Generic Link Following: The tests dynamically find the first available game or blog link, click it, and verify that the resulting URL is correct and the page body loads successfully. • Header Check: It does not strictly require a on game pages, accommodating custom game engines that might use a full-screen canvas.
---	--